



Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software
Ficha: 3134555

**Creación de la estructura de la BD y aplicación de restricciones
(GA6-220501096-AA2-EV02)**

APRENDIZ
Johan Mauricio Sánchez Gaviria

INSTRUCTOR
Nelson Ivan Guzmán

OCTUBRE 2025

Resumen

Se construye la base de datos “base de datos sena” enfocada en la creación de estructura y la aplicación de restricciones. El diseño incorpora llaves primarias y foráneas, restricciones UNIQUE, índices de apoyo y validaciones de negocio implementadas mediante TRIGGERS compatibles con XAMPP. Se incluyen datos mínimos de prueba, pruebas negativas comentadas y consultas de verificación.

Objetivo

Implementar la estructura de la base de datos y sus restricciones UNIQUE, FK, índices y validaciones de negocio, garantizando integridad referencial y reglas de dominio según la evidencia del trabajo.

1. Diseño lógico y restricciones

Esquema propuesto:

- aprendices(id_aprendiz, dni, nombres, apellidos, telefono, email, estado, fecha_registro)
- instructores(id_instructor, nombres, apellidos, email, especialidad, activo)
- cursos(id_curso, codigo, nombre, descripcion, cupo_maximo, id_instructor, fecha_inicio, fecha_fin)
- matriculas(id_matricula, id_aprendiz, id_curso, fecha_matricula, estado)

Relaciones: instructores 1N cursos; aprendices NM cursos por medio de matriculas.

2. Estructura de la BD y aplicación de restricciones

2.1 Creación de base y tablas DDL

```
DROP DATABASE IF EXISTS base de datos sena;
```

```
CREATE DATABASE base de datos sena CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_general_ci;
```

```
USE base de datos sena;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

```
CREATE TABLE aprendices (
```

```
    id_aprendiz INT AUTO_INCREMENT PRIMARY KEY,
```

```
    dni VARCHAR(20) NOT NULL,
```

```
    nombres VARCHAR(60) NOT NULL,
```

```
    apellidos VARCHAR(60) NOT NULL,
```

```
    telefono VARCHAR(20) NULL,
```

```
    email VARCHAR(120) NOT NULL,
```

```
    estado ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT
```

```
'ACTIVO',
```

```
    fecha_registro TIMESTAMP NOT NULL DEFAULT
```

```
CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE instructores (
```

```
id_instructor INT AUTO_INCREMENT PRIMARY KEY,  
  
nombres    VARCHAR(60) NOT NULL,  
  
apellidos  VARCHAR(60) NOT NULL,  
  
email      VARCHAR(120) NOT NULL,  
  
especialidad VARCHAR(80) NOT NULL,  
  
activo     TINYINT(1) NOT NULL DEFAULT 1  
  
) ENGINE=InnoDB;
```

```
CREATE TABLE cursos (  
  
id_curso    INT AUTO_INCREMENT PRIMARY KEY,  
  
codigo      VARCHAR(20) NOT NULL,  
  
nombre      VARCHAR(100) NOT NULL,  
  
descripcion VARCHAR(255) NULL,  
  
cupo_maximo INT NOT NULL,  
  
id_instructor INT NOT NULL,  
  
fecha_inicio DATE NOT NULL,  
  
fecha_fin    DATE NOT NULL  
  
) ENGINE=InnoDB;
```

```
CREATE TABLE matriculas (  
  
id_matricula INT AUTO_INCREMENT PRIMARY KEY,  
  
id_aprendiz  INT NOT NULL,  
  
id_curso     INT NOT NULL,
```

```
    fecha_matricula DATE NOT NULL DEFAULT CURRENT_DATE,  
    estado  
  
    ENUM('INSCRITO','CANCELADO','APROBADO','REPROBADO') NOT  
    NULL DEFAULT 'INSCRITO'  
  
    ) ENGINE=InnoDB;
```

2.2 Restricciones UNIQUE, FK e índices

UNIQUE

```
ALTER TABLE aprendices ADD CONSTRAINT uq_aprendices_dni
```

```
UNIQUE (dni);
```

```
ALTER TABLE aprendices ADD CONSTRAINT uq_aprendices_email
```

```
UNIQUE (email);
```

```
ALTER TABLE instructores ADD CONSTRAINT uq_instructores_email
```

```
UNIQUE (email);
```

```
ALTER TABLE cursos ADD CONSTRAINT uq_cursos_codigo UNIQUE
```

```
(codigo);
```

```
ALTER TABLE matriculas ADD CONSTRAINT uq_matricula UNIQUE
```

```
(id_aprendiz, id_curso);
```

FOREIGN KEYS

```
ALTER TABLE cursos
```

```
ADD CONSTRAINT fk_cursos_instructor
```

```
FOREIGN KEY (id_instructor) REFERENCES instructores(id_instructor)
```

```
ON UPDATE CASCADE ON DELETE RESTRICT;
```

```
ALTER TABLE matriculas

ADD CONSTRAINT fk_matriculas_aprendiz

FOREIGN KEY (id_aprendiz) REFERENCES aprendices(id_aprendiz)

ON UPDATE CASCADE ON DELETE RESTRICT,

ADD CONSTRAINT fk_matriculas_curso

FOREIGN KEY (id_curso) REFERENCES cursos(id_curso)

ON UPDATE CASCADE ON DELETE RESTRICT;
```

ÍNDICES

```
CREATE INDEX ix_aprendices_estado ON aprendices(estado);

CREATE INDEX ix_matriculas_estado ON matriculas(estado);

CREATE INDEX ix_cursos_fecha ON cursos(fecha_inicio, fecha_fin);
```

2.3 Validaciones de negocio con TRIGGER

```
DELIMITER $$

CREATE TRIGGER trg_cursos_bi

BEFORE INSERT ON cursos

FOR EACH ROW

BEGIN

    IF NEW.cupo_maximo < 1 OR NEW.cupo_maximo > 60 THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='cupo_maximo fuera de

rango (1..60)';

    END IF;
```

```
IF NEW.fecha_fin < NEW.fecha_inicio THEN

    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='fecha_fin no puede ser
anterior a fecha_inicio';

END IF;

END$$
```

```
CREATE TRIGGER trg_cursos_bu

BEFORE UPDATE ON cursos

FOR EACH ROW

BEGIN

    IF NEW.cupo_maximo < 1 OR NEW.cupo_maximo > 60 THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='cupo_maximo fuera de
rango (1..60)';

    END IF;

    IF NEW.fecha_fin < NEW.fecha_inicio THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='fecha_fin no puede ser
anterior a fecha_inicio';

    END IF;

END$$
```

```
CREATE TRIGGER trg_email_aprendiz_bi

BEFORE INSERT ON aprendices

FOR EACH ROW
```

```
BEGIN

    IF INSTR(NEW.email, '@') = 0 OR INSTR(NEW.email, '.') = 0 THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='email inválido
(formato básico)';

    END IF;

END$$
```

```
CREATE TRIGGER trg_email_aprendiz_bu

BEFORE UPDATE ON aprendices

FOR EACH ROW

BEGIN

    IF INSTR(NEW.email, '@') = 0 OR INSTR(NEW.email, '.') = 0 THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='email inválido
(formato básico)';

    END IF;

END$$
```

```
CREATE TRIGGER trg_email_instructor_bi

BEFORE INSERT ON instructores

FOR EACH ROW

BEGIN

    IF INSTR(NEW.email, '@') = 0 OR INSTR(NEW.email, '.') = 0 THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='email inválido
```


(formato básico)';

END IF;

END\$\$

CREATE TRIGGER trg_email_instructor_bu

BEFORE UPDATE ON instructores

FOR EACH ROW

BEGIN

IF INSTR(NEW.email, '@') = 0 OR INSTR(NEW.email, '.') = 0 THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='email inválido

(formato básico)';

END IF;

END\$\$

DELIMITER ;

3. Datos mínimos y pruebas

3.1 Carga mínima de datos

INSERT INTO instructores (nombres, apellidos, email, especialidad, activo)

VALUES

('Ana','Gómez','ana.gomez@sena.edu.co','Bases de Datos',1),

('Luis','Ramírez','luis.ramirez@sena.edu.co','Programación Web',1);

INSERT INTO aprendices (dni, nombres, apellidos, telefono, email, estado)

VALUES

```
('1012345678','Johan','Sánchez','3001112233','johan.sanchez@example.com','ACTIVO'),  
( '1012345679','María','Pérez','3002223344','maria.perez@example.com','ACTIVO'  
);
```

```
INSERT INTO cursos (codigo, nombre, descripcion, cupo_maximo, id_instructor,  
fecha_inicio, fecha_fin) VALUES  
( 'SQL101','Fundamentos de SQL','Curso básico de SQL',30,1,'2025-10-01','2025-  
11-15'),  
( 'WEB201','Frontend Básico','HTML, CSS y JS',25,2,'2025-10-05','2025-11-30');
```

```
INSERT INTO matriculas (id_aprendiz, id_curso, estado) VALUES  
(1,1,'INSCRITO'),  
(1,2,'INSCRITO'),  
(2,1,'INSCRITO');
```

3.2 Pruebas negativas

cupo_maximo inválido

```
INSERT INTO cursos (codigo, nombre, cupo_maximo, id_instructor,  
fecha_inicio, fecha_fin)  
VALUES ('BAD001','Cupo inválido',0,1,'2025-10-01','2025-10-10'); -- ERROR
```

fecha_fin < fecha_inicio

```
INSERT INTO cursos (codigo, nombre, cupo_maximo, id_instructor,  
fecha_inicio, fecha_fin)  
VALUES ('BAD002','Fechas inválidas',10,1,'2025-11-10','2025-10-10'); --  
ERROR
```

email inválido (aprendiz)

```
INSERT INTO aprendices (dni, nombres, apellidos, email)  
VALUES ('999','Test','Error','correo-sin-arroba'); -- ERROR
```

Duplicado UNIQUE en matrícula

```
INSERT INTO matriculas (id_aprendiz,id_curso) VALUES (1,1); -- ERROR
```

4. Consultas de verificación

Cursos con instructor

```
SELECT c.id_curso, c.codigo, c.nombre, c.cupo_maximo,  
       CONCAT(i.nombres,' ',i.apellidos) AS instructor,  
       c.fecha_inicio, c.fecha_fin  
FROM cursos c JOIN instructores i ON i.id_instructor=c.id_instructor  
ORDER BY c.id_curso;
```

Matrículas con detalle

```
SELECT m.id_matricula, a.dni, CONCAT(a.nombres,' ',a.apellidos) AS aprendiz,  
       c.codigo, c.nombre AS curso, m.estado, m.fecha_matricula
```

```
FROM matriculas m  
JOIN aprendices a ON a.id_aprendiz=m.id_aprendiz  
JOIN cursos c ON c.id_curso=m.id_curso  
ORDER BY m.id_matricula;
```

Referencias

Guía 6 – Evidencia GA6-220501096-AA2-EV02 (documento interno SENA).

MySQL 8.0 / MariaDB 10.x Reference Manual. (s. f.).

<https://dev.mysql.com/doc/>; <https://mariadb.com/kb/en/documentation/>